Tecnicatura Universitaria en Inteligencia Artificial

# Procesamiento de Imágenes I

Informe Trabajo Práctico


**Integrantes del Grupo:**

Castagna, Emiliano - C-7553/1
Gomez, Micaela - G-6088/7
Jacob, Zahir - J-0625/4

## INTRODUCCIÓN

El trabajo consta de dos problemas principales: el primero consiste en la implementación de una ecualización local de histograma para revelar detalles ocultos en una imagen, y el segundo aborda el procesamiento de imágenes de exámenes "múltiple choice" para su corrección automática y validación de campos

## Ejercicio 1 - Ecualización Local de Histograma

El objetivo es implementar una técnica de ecualización local del histograma. A diferencia de la global, este método consiste en definir una ventana de tamaño MxN que se desplaza píxel por píxel. En cada ubicación, se calcula el histograma de la ventana y se ecualiza, pero el resultado se utiliza únicamente para mapear la intensidad del píxel central. Esto permite resaltar detalles en zonas donde la intensidad de los objetos es muy similar al fondo local, algo que la ecualización global no logra resolver adecuadamente. Posteriormente, se debe analizar la influencia del tamaño de la ventana.

## Método Utilizado

En el algoritmo *Problema_1.py*, se implementó una función que define una ventana de tamaño MxN que se desplaza sobre la imagen.

```
def ecualizacion_local(imagen: np.ndarray, tamaño: int):
    '''
        Recibe dos cosas: la imagen y el tamaño de la ventana.
        Esa ventana es un cuadrado que va a ir recorriendo
        toda la imagen.
    '''
    '''
        Se agregan bordes artificiales copiando los valores del borde real
        así la ventana siempre puede existir completa.
    '''
```

En cada posición, se toma la región y se calcula su histograma, aplicando ecualización local con *cv2.equalizeHist()* para mejorar la distribución de intensidades. Luego, se reemplaza solo el valor del píxel central.

```
for fila in range(filas):
    for columna in range(columnas):
        ventana = imagen_extendida[fila: fila + tamaño, columna: columna + tamaño]
        ventana_ecualizada = cv2.equalizeHist(ventana)
        resultado[fila, columna] = ventana_ecualizada[margen, margen]
```

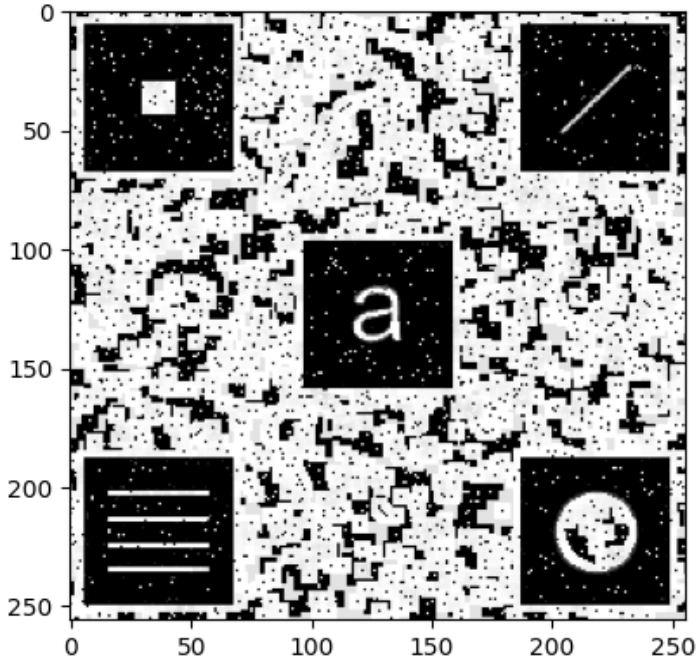
Durante el desarrollo surgió un inconveniente con los bordes de la imagen. Al acercarse a los extremos, la ventana no podía completarse correctamente por la falta de píxeles vecinos. Para solucionarlo, decidimos agregar bordes artificiales utilizando la función *cv2.copyMakeBorder()*, con el parámetro *cv2.BORDER_REPLICATE*.

```
margen = tamaño // 2
imagen_extendida = cv2.copyMakeBorder(imagen, margen, margen, margen, margen,cv2.BORDER_REPLICATE)
```

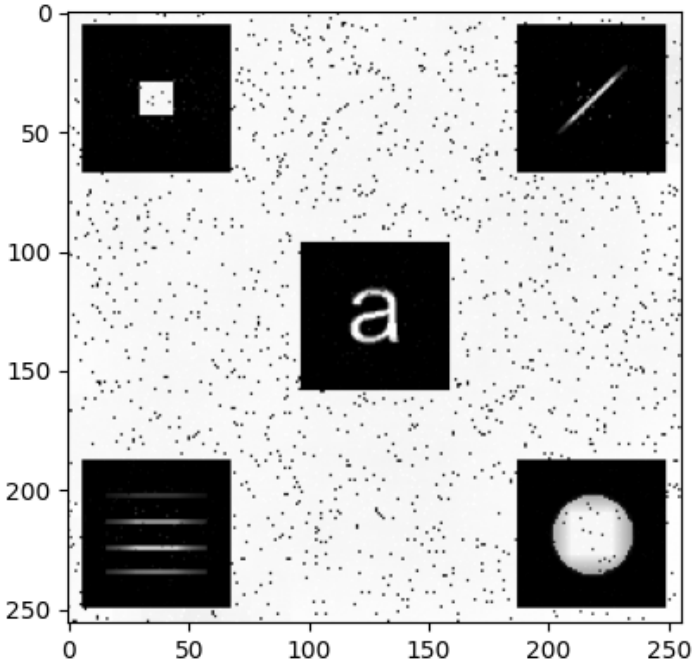
Con esto, replicamos los bordes hacia afuera permitiendo que la ventana pueda posicionarse sobre todos los píxeles originales manteniendo su tamaño, logrando procesar la imagen completa sin recortes en los bordes.

Al probar el código con diferentes tamaños de ventana (7x7, 25x25 y 45x45), pudimos observar cómo el tamaño influye en el resultado final:

- Con ventanas chicas (7x7) se obtiene mayor calidad de detalles finos de la imagen, pero como se hace mucho más evidente el ruido de fondo.




- Con ventanas más grandes (45x45) el resultado global se percibe más suave y estable, pero los detalles más pequeños y sutiles pierden intensidad al promediarse con un fondo mucho más grande.



## Ejercicio 2 - Corrección de Múltiple Choice

Este problema nos plantea el desafío de corregir exámenes tipo "múltiple choice" de 10 preguntas de manera automática a partir de imágenes. El algoritmo debe identificar las marcas de cada respuesta y compararlas con una plantilla esperada para calificar. Además, es necesario validar los campos ingresados a mano en el encabezado (*Name*, *Date*, *Class*) bajo ciertas reglas (cantidad de palabras/caracteres). Finalmente, se debe generar un panel o imagen de salida ordenando a los alumnos en "Aprobados" y "Desaprobados" extrayendo los recortes visuales de sus nombres.

## Método Utilizado

En el script *Problema_2.py* se utilizaron técnicas de segmentación morfológica, análisis de componentes conectadas y contornos.

1. **Binarización y Detección de la Grilla (Líneas):**

   Como primer paso, la imagen en escala de grises es invertida y binarizada (`cv2.threshold` con `THRESH_BINARY_INV`, umbral 190) para que los trazos de los párrafos y líneas de las tablas figuren como blancos sobre negro brillante.

```
imagen_binaria = cv2.threshold(imagen, 190, 1, cv2.THRESH_BINARY_INV)
```

   Para detectar la grilla de celdas, sumamos la intensidad de píxeles verticalmente (eje X) y horizontalmente (eje Y) usando proyecciones de numpy (`np.sum`).

```
suma_columnas = np.sum(imagen_binaria, axis=0)
suma_filas = np.sum(imagen_binaria, axis=1)
```

   Establecimos umbrales para descartar el texto y nos quedamos solo con las líneas reales continuas. Esto nos brindó los índices para recortar celdas individualmente más adelante.

2. **Extracción y Validación del Encabezado:**

   Una vez segmentado el tercio superior como "encabezado", se aplicó *cv2.connectedComponentsWithStats* para aislar las manchas correspondientes a la escritura del usuario.

```
campos = []
cantidad, etiquetas, stats, centroides = cv2.connectedComponentsWithStats(encabezado, 8, cv2.CV_32S)

for stat in stats:
    ancho = stat[2]
    if 50 < ancho < 200:
        x = stat[0]
        y = stat[1]
        w = stat[2]
        campos.append(encabezado[y - 20:y - 2, x:x + w])
```

   Filtrando por áreas específicas (anchos entre 50 y 200 px), el script puede aislar exitosamente los recuadros donde se escribió la información.

Para validar las restricciones de formato (palabras y longitud), implementamos un algoritmo heurístico que recorre las componentes conectadas de un campo, asumiendo que cada una representa una letra, y monitoreando las separaciones espaciales en el eje X: si la distancia entre caracteres es mayor a un umbral (mayor a 15), se asume que existe un espacio, contabilizando así otra palabra. Esta técnica resultó muy efectiva para validad que nombre tenga al menos dos palabras o la fecha un único bloque. Adicionalmente, guardamos el recorte de píxeles visuales correspondiente al campo "Name" para armar luego la placa final.

3. **Análisis de las Celdas y Reconocimiento de Respuestas:**

Habiendo segmentado la grilla en el paso 1, cada pregunta fue individualizada. Al buscar la respuesta escrita con *connectedComponentsWithStats*, se analizan las zonas donde habitan marcas y se pasa esa sub-zona por un proceso de identificación de letra ("A", "B", "C", "D").

Decidimos realizar este reconocimiento analizando la topología de la letra inscrita apoyándonos en los contornos (*cv2.findContours*):

- Si posee 1 solo contorno externo/interno -> clasificamos letra "C".
- Si posee 3 contornos (dos agüjeros internos de las pancillas de la letra B, más el entorno) -> clasificamos letra "B".
- Si posee 2 contornos internos/externos, el dilema es entre la "A" y la "D". Aquí se resolvió calculando y verificando el área paramétrica (*cv2.contourArea*). Si el área cuadra alrededor de ~23.0 es "A", contrario es "D".

Este método directo nos previno de usar plantillas (Template Matching) complejas, que en escritura variable a mano podría fallar debido a escala o rotación.

4. **Corrección y Panel de Resultados:**

Las letras devueltas son contrastadas contra un diccionario solución. Un alumno necesita al menos 60% aprobado para pasar. Mediante *OpenCV*, se crea un lienzo nuevo en blanco gigante en el cual reubicamos los recortes crudos de cada "*Name*" escaneado ("Juan", "Pedro", etc). Se les dibuja un recuadro verde y se los apila en la columna "APROBADOS" si pasaron, o recuadro rojo en "DESAPROBADOS" si fallaron, logrando la estructura requerida.

## CONCLUSIONES OBTENIDAS

El Trabajo Práctico evidenció fuertemente la importancia del procesamiento local y el filtrado sobre la imagen dependiendo del resultado buscado.

En el **Ejercicio 1** quedó muy claro el sacrificio entre suavidad de imagen con ventanas grandes con bajo ruido, y ventanas pequeñas con detalles recuperables pero que amplifica el ruido. El uso de bordes artificiales es una herramienta vital para aplicar núcleos o ventanas sin alterar topologías ni medidas en la imagen final.

Por otro lado, en el **Ejercicio 2**, abordamos un flujo de procesamiento morfológico completo. Entendimos la importancia de realizar una correcta binarización previa, ya que si esta falla las proyecciones de filas/columnas para extraer grillas no funcionan, rompiendo la detección de campos y letras.

También se exploró el uso de las funciones de componentes conectadas (*cv2.connectedComponentsWithStats*) para tratar el análisis documental, permitiendo hacer recuento de palabras midiendo píxel por píxel el "*white space*" real entre letras o discriminar carácteres solo analizando sus hoyos / topología y contornos de *OpenCV* en escritura simple.